

The Alberta Buck - Notes Flow (DRAFT v0.1)

How One BUCK Transfer Becomes N Unlinkable Notes (Step by Step)

Perry Kundert

2026-04-20

1 Abstract

This document is the *narrative* companion to alberta-buck-notes.org (architectural overview), alberta-buck-proofs.org (formal correctness), and alberta-buck-ethereum.org (Solidity surface). It walks one concrete example – Alice mints a 1,200-BUCK transfer into six notes of denominations [1000, 100, 25, 25, 25, 25] – end-to-end through the Phase-6 Groth16-verified mint pipeline, naming every actor, every artifact, and every check. Where the example bumps up against an open question (Phase-7 spend, mint-circuit range bounds), the gap is flagged inline rather than papered over.

The aim is to show, by walking the path, that the system delivers five concrete guarantees:

1. **Atomic conservation** – if the transaction succeeds, the BUCK balance leaves Alice exactly once and matches the public total of the new notes. If the proof fails, no BUCK moves and no commitments are appended.
2. **Issuer-bound, recipient-blinded artifacts** – the public commitments name the issuer (`msg.sender`) but reveal nothing about per-note values or recipients.
3. **Per-note disclosure to recipients only** – each recipient receives an *opening* off-chain that lets them reconstruct exactly their own note’s value and the issuer’s verifiable identity, plus the fact that N siblings exist whose hidden values sum to `totalFace` – and nothing more.
4. **Offline receipt reconstruction** – given only the mint transaction ID, anyone holding an opening can independently verify the note against the chain, with no help from Alice or any indexer.
5. **Adversarial-prover soundness** – even if Alice runs a maliciously patched EVM and an arbitrarily hostile witness generator, the honest validators executing the deployed `Notes.sol` and `MintGroth16Verifier.sol` reject every batch that does not satisfy the mint relation. The on-chain truth is what governs.

The remaining caveats (a known field-overflow gap on `v[i]` and the deferred spend circuit) are addressed at the end and cross-referenced into the proofs and implementation docs.

2 Cast of Characters

The example uses concrete names so the message flow is easy to track:

- **Alice** – the issuer. She holds 1,200 BUCK and wants to break it into six notes. She is registered with the IdentityRegistry so that her `msg.sender` address resolves to a cryptographically attested identity.
- **Bob, Carol, Dave, Eve, Frank, Grace** – six recipients. Bob will get the 1,000-BUCK note; Carol the 100-BUCK note; Dave/Eve/Frank/Grace one 25-BUCK note each. Recipients do *not* need to be on-chain known to Alice – the recipient identity binding is a field inside the commitment, not an on-chain registration step at mint.
- **The honest EVM** – every Ethereum validator running stock client software. This is the only EVM whose verdict matters. It executes the deployed `Notes.sol`, `MintGroth16Verifier.sol`, `MintVerifierAdapter.sol`, and `Buck.sol` bytecode.
- **Alice’s hostile EVM (Eve-EVM)** – a fictional entity Alice can run locally. She may patch any contract logic she wants, run any witness generator, and produce any proof bytes. The point of the design is that nothing she runs locally affects what the honest EVM accepts.
- **A passive observer (Mallory-the-watcher)** – someone scanning the public chain after the fact, trying to learn *which* note went to *whom* for *how much*. Mallory has full chain history and unlimited compute.

3 The Promise

By the end of the mint transaction Alice will have:

- A single Ethereum transaction `T_mint` on-chain whose effects are:
 - 1,200 BUCK debited from Alice and credited to the `Notes` pool address.
 - Six opaque `uint256` commitments `[cm_0..cm_5]` appended to `Notes.commitments[]`.
 - One `Minted(issuer=Alice, totalFace=1200, startIndex=K, count=6)` event.
 - One `Transfer(Alice, Notes, 1200)` ERC-20 event.
 - One Groth16 proof in `calldata`, accepted by the on-chain verifier.
- Six off-chain artifacts, one per recipient, each containing the *opening* of exactly one commitment plus the mint-transaction reference.

The key invariant: looking at the chain alone, Mallory sees that Alice escrowed 1,200 BUCK and posted six hashes. She cannot tell which hash hides which value, who any recipient is, or even whether two of the recipients are the same person. Looking at his off-chain artifact, Bob sees his 1,000 BUCK is real, was minted by Alice in `T_mint`, and that five other notes exist whose hidden values sum to 200 BUCK – but he does not learn what those values are or who holds them.

4 What’s Inside a Note (Conceptually)

A "note" is just a five-tuple known by Alice and the recipient:

```
opening_i = ( flavor_i, v_i, rho_i, idHash_i, predicate_i )
cm_i      = Poseidon5( flavor_i, v_i, rho_i, idHash_i, predicate_i )
```

- **flavor_i** – which note variant (A1 identified cheque, A2 private cheque, B1 bearer cheque); see Notes for the menu.
- **v_i** – the face value in BUCK base units (18-decimal scaled integer).
- **rho_i** – a fresh, uniformly random nonce. This is the source of unlinkability: swapping **rho_i** changes **cm_i** to a uniformly fresh hash even if every other field is identical.
- **idHash_i** – a Poseidon-hashed handle on the recipient’s identity (e.g. a Poseidon-hash of the recipient’s registered ElGamal credential, or zero for a pure bearer note).
- **predicate_i** – a domain-separating tag for any optional spending condition (escrow, time-lock, etc.). Zero in the simplest case.

The Poseidon-5 hash gives the commitment two cryptographic properties that make the whole flow work:

- **Hiding.** Without **rho_i**, the commitment leaks no information about the other fields. Mallory sees **cm_i** and gains nothing about **v_i**, **idHash_i**, or **flavor_i**.
- **Binding.** No prover, hostile or honest, can find two distinct openings hashing to the same **cm_i** (collision-resistance of Poseidon on BN254, modelled as a random oracle in alberta-buck-proofs.org Part IV).

The on-chain contract stores only **cm_i**. Openings live exclusively off-chain.

5 The Mint, Step by Step

5.1 Step 0: Pre-flight (Off-Chain, In Alice’s Wallet)

Alice’s wallet has done all the slow work before any transaction is broadcast:

- She owns 1,200 BUCK on-chain.
- She has decided on the denomination split: [1000, 100, 25, 25, 25, 25]. N=6, but the Phase-6 circuit is pinned at N=2 (see Phase 6); for the example we proceed *as if* a future Mint(6) circuit were already in place. The structure of every step is identical – only the circuit constant changes.
- She has chosen an **idHash_i** for each recipient (typically a Poseidon hash of the recipient’s IdentityRegistry-bound ElGamal credential, derived once at out-of-band introduction time).
- Her CSPRNG produces six fresh **rho_i** nonces.

She now has six opening tuples in memory.

5.2 Step 1: Build Commitments

For $i = 0..5$ her wallet computes

```
cm_i = Poseidon5( flavor_i, v_i, rho_i, idHash_i, predicate_i )
```

This is pure off-chain Poseidon arithmetic in `alberta_buck.wallet.notes`. Six `uint256` field elements drop out. Their order in the eventual on-chain array is fixed by Alice’s local list – the contract preserves whatever order she submits.

5.3 Step 2: Generate the Groth16 Proof

Alice’s wallet runs `scripts/snark/prove_mint.js` (or the production equivalent). The witness consists of:

- Public inputs: `totalFace = 1200e18`, `cm = [cm_0, cm_1, cm_2, cm_3, cm_4, cm_5]`.
- Private witnesses: `flavor[i]`, `v[i]`, `rho[i]`, `idHash[i]`, `predicate[i]` for each `i`.

The `mint.circom` circuit checks two relations across the witness:

1. **Per-note opening.** For each `i`, `cm[i] === Poseidon5(flavor[i], v[i], rho[i], idHash[i], predicate[i])`. Six Poseidon evaluations, six equality constraints.
2. **Sum conservation.** `sum(v[0]..v[5]) === totalFace`. One linear constraint.

The witness is processed by snarkjs’s Groth16 prover. A few hundred milliseconds later the wallet has a fixed-size proof object: (`pi_A: G1`, `pi_B: G2`, `pi_C: G1`). Encoded as ABI-static (`uint256[2]`, `uint256[2][2]`, `uint256[2]`) it is exactly 256 bytes.

Observation. The Groth16 proof reveals *nothing* about the witness. Soundness guarantees that no `proofBytes` can satisfy the verification equation unless a witness exists; zero-knowledge guarantees that the verification equation reveals nothing about which witness was used. Mallory, even with the proof in hand, cannot back out `v[i]`, `rho[i]`, `idHash[i]`, `flavor[i]`, or `predicate[i]`.

5.4 Step 3: Submit Notes.mint On Chain

Alice now broadcasts a single transaction `T_mint` calling `notes.mint(proofBytes, [cm_0..cm_5], 1200e18)`;

The honest EVM walks `Notes.mint`:

```
require(cms.length > 0, "Notes: empty mint");
require(mintVerifier.verifyMint(proof, totalFace, cms, msg.sender),
    "Notes: bad mint proof");
require(buck.transferFrom(msg.sender, address(this), totalFace),
    "Notes: transfer failed");
// ... append cms[i] to commitments[], assert no duplicates,
// emit Minted(...).
```

Three things happen in strict order, and **all of them are atomic** (one transaction, one EVM state transition):

a. The `MintVerifierAdapter` unpacks the 256-byte `proofBytes` into Groth16 form, builds the public-signal vector `[totalFace, cm_0..cm_5]`, and invokes the `MintGroth16Verifier`. If the verifier returns false, the entire transaction reverts – nothing is written, no BUCK moves. b. The BUCK ERC-20’s `transferFrom` debits `totalFace` from Alice and credits the `Notes` pool. This is a normal allowance-driven pull, gated by Alice’s prior `approve(notes, 1200e18, ...)`. Because the `Notes` pool is registered as *system-public* in the `IdentityRegistry`, the bilateral identity check on the transfer leg succeeds without a Chaum-Pedersen receipt. c. For each `cm_i`: the contract checks `!commitmentExists[cm_i]`, marks it true, appends to `commitments[]`, emits `Appended(cm_i, startIndex+i)`. Finally a single `Minted(Alice, 1200e18, startIndex, 6)` event closes the call.

If any of (a), (b), or (c) fails – bad proof, insufficient BUCK, missing approval, duplicate commitment – the entire transaction reverts and the chain state is indistinguishable from no submission at all. **Alice gets one shot per transaction**; she cannot retry a partial state.

5.5 Step 4: Off-Chain Note Delivery

After `T_mint` confirms, Alice transmits each opening to its intended recipient by any out-of-band channel: an encrypted message, a printed QR code, a USB stick. The payload to recipient `i` is:

```
note_artifact_i = {
    txHash:      T_mint,                # Ethereum transaction ID
    index:       startIndex + i,        # position in Notes.commitments[]
    cm:          cm_i,                  # for cross-checking against the chain
    opening:     ( flavor_i, v_i, rho_i, idHash_i, predicate_i ),
    meta:        { issuer: Alice's IdentityRegistry handle, ... },
}
```

The chain itself never sees `opening`. `cm_i` is already on-chain; the artifact includes it only as a convenience so the recipient does not have to recompute it before locating his entry in `commitments[]`.

5.6 Step 5: Recipient Verification (Offline, Independent)

Bob receives his artifact for the 1,000-BUCK note. Without trusting Alice and without any help from an indexer, he runs:

1. **Recompute and check.** Bob computes `cm_check = Poseidon5(flavor, 1000e18, rho, idHash, predicate)` and asserts `cm_check == cm`. If this fails, Alice lied about the opening; Bob discards the artifact and demands a real one. (Binding of Poseidon makes substitution impossible.)
2. **Look up the mint transaction.** Bob reads `T_mint` on-chain, parses the call data into `(proofBytes, cms[], totalFace, msg.sender)`.
3. **Confirm `msg.sender` is Alice.** Cross-check `msg.sender` against the IdentityRegistry: it must resolve to Alice's PS-credentialed identity. If Bob wanted an A1 identified cheque, the issuer field of his opening must match.
4. **Confirm membership.** Read `Notes.commitments(index)` via an `eth_call` and assert it equals `cm`. Bob now has chain-anchored evidence that `cm` is officially a commitment in the pool.
5. **Confirm batch totals.** Read `totalFace` from the call data. The chain's acceptance of `T_mint` is itself a proof that the Groth16 verifier accepted – which means `sum(v[0..5]) == totalFace` holds in the witness. Bob does not need to re-verify the proof; the EVM already did. But if he wants belt-and-suspenders, he can re-run `MintGroth16Verifier.verify(proofBytes, [totalFace, cms...])` locally against the same on-chain bytecode, and get the same accept.

After (1)-(5) Bob knows:

- His note encodes exactly 1,000 BUCK. (Step 1.)
- It was minted in `T_mint` by Alice. (Steps 2, 3.)
- It is one of exactly six commitments produced by that mint, the others being `cms[0..5] \ cm`. (Step 4.)

- The hidden values of the other five sum to exactly $1200 - 1000 = 200$ BUCK. (Step 5
 - the per-note opening relation: he knows *his* value, the chain attests to the *sum*.)
- He learns nothing else about the other notes – not their individual values, not their recipients, not their flavors. Each `cm_j` is hiding under a fresh `rho_j` he does not have.

This is the unlinkability promise made concrete: each recipient can pin down their own slice and the batch total, without learning anything about siblings.

6 Why Alice Cannot Cheat (Even With a Hostile EVM and Prover)

Alice runs whatever software she wants on her laptop. She can fork the `snarkjs` witness generator, patch the `circom` compiler, run `Anvil` with a malicious `MintGroth16Verifier` that always accepts. None of it matters, because the truth on the public chain is determined by what *the honest validators* run. Three pillars prop up the soundness story:

6.1 Pillar 1: The On-Chain Verifier Is Fixed Code

`MintGroth16Verifier.sol` was emitted from the `Groth16` verification key produced by the trusted setup (Phase 9 for production; dev-only entropy in `scripts/snark/setup.sh` today). Its bytecode is deployed once and immutable. Validators run whatever bytecode sits at that address; Alice cannot tamper with it without consensus-breaking the chain.

Knowledge soundness of `Groth16` means: for every `proofBytes` the verifier accepts on public inputs `[totalFace, cm_0..cm_N-1]`, there exists a witness (`flavor[i]`, `v[i]`, `rho[i]`, `idHash[i]`, `predicate[i]`)_i such that:

- `cm[i] = Poseidon5(flavor[i], v[i], rho[i], idHash[i], predicate[i])` for all `i`
- `sum(v[i]) = totalFace`

Alice is free to *find* such a witness any way she likes – she’s the prover. But she cannot find a witness for a *false* statement; that’s what soundness prohibits.

6.2 Pillar 2: BUCK Escrow Is Inside the Same Atomic Call

The proof check, the BUCK pull, and the commitment append all live in the same `Notes.mint` function. An EVM transaction is all-or-nothing: the only two outcomes are "the entire call’s state changes commit" or "the entire call’s state changes revert".

This means Alice cannot, for instance:

- Verify a proof, then refuse to pay – the verifier acceptance and the `transferFrom` live in the same call, gated by the same `require`.
- Pay 1,200 BUCK but then get away with appending only one `cm` and pocketing the delta – the loop appending commitments runs after `transferFrom`, in the same call, and a revert at any point unwinds the transfer too.
- Trick the contract into appending a duplicate `cm` – `commitmentExists[cm]` is checked per item.

6.3 Pillar 3: Public Inputs Are Sealed Into the Proof

The Groth16 verifier consumes the public-signal vector `[totalFace, cm_0, ..., cm_N-1]`. The proof was generated against the specific values Alice committed to before submission. If Alice tries to submit the proof with a different `totalFace` or with the `cms` permuted, the verifier rejects.

This is the bedrock of the test suite in `test/MintVerifier.t.sol` – four explicit tamper paths (`totalFace + 1`, `cm[0] ^ 1`, wrong batch size, malformed proof bytes) each revert with `Notes: bad mint proof` or an ABI decode error. Soundness is not a proof of *absence* of attack, but every documented attack surface is closed by a test that triggers a verifier reject.

6.4 Putting the Pillars Together: The Adversarial Walk

Suppose hostile-Alice tries to mint two **contradictory** batches that both claim to be the canonical breakdown of her `T_mint` transfer. She would need:

- Two different `proofBytes` and/or `cms[]`, both passing the on-chain verifier.

That requires either:

- Two valid Groth16 witnesses for the same public inputs (allowed – but they yield the same `cms`, so they are not *contradictory*), or
- Two valid Groth16 witnesses with *different* public inputs. Allowed too, but each one demands its own `transferFrom` for its own `totalFace` – **that** is the duplicate-spend problem ERC-20 solves. Alice cannot transfer the same 1,200 BUCK twice without holding 2,400 BUCK to start with.

So a single 1,200-BUCK escrow yields exactly one accepted batch. The only freedom Alice has is *which* batch she lays down – which denominations and which recipients – and that freedom is hers to keep, by design.

What about hostile-Alice trying to mint *more* commitment value than she escrowed? She would need a witness whose `sum(v[i])` equals the integer `totalFace` *that the EVM sees*, but whose individual `v[i]` values *actually* sum to more. This is Open Question 1 in the proofs document: in `F_r` with `r ~ 2^254` and no `Num2Bits` range check on `v[i]`, the constraint `sum(v[i]) === totalFace` is satisfiable by witnesses whose integer sum differs from `totalFace` by a multiple of `r`. Currently this attack is *dormant* because Phase 2 has no spend path – Alice would have to escrow `totalFace` up front and then never be able to release more than `totalFace` back, since no spend mechanism exists. *This dormant gap closes the moment Phase 7 ships*, and the fix (`Num2Bits(128)` per `v[i]`) is tracked as a Phase-7 prerequisite in alberta-buck-ethereum.org.

7 Why Outside Observers Cannot Link

Mallory has full read access to the chain after `T_mint` confirms. What does she see?

`T_mint:`

```
from: Alice (a registered IdentityRegistry account)
to:    Notes (a system-public contract)
value: 0 (BUCK is ERC-20, not ETH)
data:  Notes.mint(proofBytes, [cm_0..cm_5], 1200e18)
```

```

events emitted (in order):
  Buck.Transfer(Alice, Notes, 1200e18)
  Notes.Appended(cm_0, K+0)
  ...
  Notes.Appended(cm_5, K+5)
  Notes.Minted(Alice, 1200e18, K, 6)

```

What can Mallory infer?

- That Alice (publicly) escrowed 1,200 BUCK. **Yes** – her address is `msg.sender`.
- That six new commitments entered the pool. **Yes** – she sees the indices.
- *Which* commitment is worth *how much*. **No** – each is a fresh `Poseidon5` output hidden by an independent `rho_i`. Without a `rho_i`, distinguishing the 1,000-BUCK hash from the 25-BUCK hash is computationally equivalent to inverting Poseidon on a uniform field element, modelled here as a random oracle.
- *Who* holds each note. **No** – `idHash_i` is also inside the hash.
- Whether two commitments share a recipient. **No** – two notes addressed to the same `idHash` but with distinct `rho_i` produce uniformly independent `cm_i`, so the chain shows nothing distinguishable from independent draws.

What about *linking back to a specific note artifact* that Bob later possesses? Mallory sees `cm_0..cm_5` published by Alice in `T_mint`. Bob's artifact references some `cm_i` and `T_mint`, but the artifact lives off-chain. Mallory cannot get the artifact unless Bob (or Alice) shows it to her, and the chain alone gives her no way to single out `cm_i` as "the one Bob holds" against the other five.

The remaining linkability surface is the issuer. `msg.sender` is publicly Alice, so every commitment in the batch carries a public **issuer** tag. This is intentional and desired: a recipient must be able to verify who minted his note. The privacy goal is unlinkability of recipients and amounts, not of issuers.

7.1 Spend-Time Unlinkability (Phase 7 Forward Reference)

When the spend circuit ships (Phase 7), Bob will be able to deposit his note without revealing *which* `cm` in the on-chain set he opened. The spend SNARK proves "I know an opening to *some* commitment in the published Merkle tree that hashes to this nullifier", with `cm` as a private witness rather than a public input.

That is when the unlinkability of *spend events* to *mint events* becomes a true chain-observable property. Today (Phase 6) Bob still has to point at a specific `cm` when redeeming – because no redemption path exists yet at all.

8 Atomicity, No Double-Mint, No Replay

Three properties together rule out duplicated or partial mints:

8.1 Atomicity

Already covered above: `Notes.mint` performs proof verification, BUCK pull, and commitment appends in one EVM call. Either all happen or none.

8.2 No Double-Mint From One Transfer

The transfer and the commitments are bound in the same call: there is no "pre-deposit, then mint" or "mint, then deposit later" flow that an attacker could desynchronize. Each `Notes.mint` invocation does its own `transferFrom`; each `transferFrom` debits Alice's BUCK balance once. Two minting transactions require two `transferFrom` debits, which require Alice to actually hold $2 * \text{totalFace}$ BUCK.

8.3 No Replay

A literal replay – broadcasting the exact same `T_mint` calldata in a new transaction – fails for two independent reasons:

a. **Commitment uniqueness.** `commitmentExists[cm_i]` is set true on the first append. The second submission would revert at the first `require(!commitmentExists[cm_i])` check. (Replay-of-cm with a different `rho` is structurally different and is simply a fresh mint, not a replay.) b. **Allowance and balance.** Even if the prior call's state were somehow reverted, the second mint would re-debit Alice's BUCK and re-consume her allowance to the `Notes` pool.

8.4 Cross-Chain or Cross-Deployment Replay

The `MintGroth16Verifier` verification key is fixed by the trusted-setup ceremony for that deployment. A proof generated against the dev-key in `scripts/snark/setup.sh` will not verify against a production-key contract on mainnet, and vice versa. Public inputs are also sealed into the proof: a proof valid on Sepolia for `totalFace=1200` cannot be reused on mainnet because the on-chain verifier address (and key) differ.

The only cross-context vulnerability would be a *chain-id collision* on identical verification keys, which the production setup ceremony precludes by construction.

9 Receipt Construction From the Transaction ID Alone

A practical question: months later, Bob wants to prove to Carol that his note was legitimately issued. He needs no help from Alice and no live indexer:

1. Bob fetches `T_mint` from any Ethereum archive node.
2. He decodes the call data into `(proofBytes, cms, totalFace)` using the standard `Notes.mint` ABI.
3. He notes `msg.sender` and resolves it to Alice's identity via the `IdentityRegistry` on the same chain (a single `eth_call`).
4. He locates his `cm` in `cms[]` at the index recorded in his artifact and asserts equality.
5. He recomputes `Poseidon5(flavor, v, rho, idHash, predicate) == cm` from the opening.

6. (Optional) He re-runs `MintGroth16Verifier.verify(proofBytes, [totalFace, cms...])` locally to convince himself the EVM did its job.

The "receipt" Carol receives is therefore `(T_mint, opening, idHash mapping)`. Its validity is independently checkable against the public chain. Nothing here requires ongoing cooperation from Alice – which is exactly the property a receipt should have.

10 Putting the Five Guarantees Side-by-Side

Guarantee	Mechanism
Atomic conservation	Single EVM call: proof verify, <code>transferFrom</code> , append-cms all-or-nothing
Issuer-bound, recipient-blinded	<code>msg.sender</code> is public; <code>cm_i</code> is Poseidon-hidden over <code>(flavor, v, rho, idHash, pred)</code>
Per-note disclosure (recipient only)	Off-chain opening contains exactly one note's witness fields
Offline receipt reconstruction	<code>T_mint</code> calldata + opening + on-chain Identity lookup is sufficient and self-checkable
Adversarial-prover soundness	Groth16 knowledge soundness on fixed on-chain verifier; ERC-20 balance/allowance integrity

Each row is enforced by code already in the tree: Phase 5 (`Notes.sol`, `StubMintVerifier`) for the contract surface, Phase 6 (`mint.circom`, `MintGroth16Verifier`, `MintVerifierAdapter`) for the SNARK pipeline, the `IdentityRegistry` for issuer attestation, and the formal proofs (Theorems 7-11) for the cryptographic backstop.

11 Open Questions and Forward-Looking Caveats

The example assumed properties that the Phase-6 deployment does not yet fully discharge. Each of these is tracked in greater depth elsewhere; they are listed here so the reader is not misled about the current state.

1. **Range bounds on `v[i]`.** The constraint `sum(v[i]) === totalFace` holds in `F_r` and the circuit lacks per-`v` range checks. An attacker today could craft a witness whose integer-sum overshoots `totalFace` by a multiple of `r` – but cannot extract that surplus, because Phase 2 has no spend path. The fix (`Num2Bits(128)` per `v[i]`) is a Phase-7 prerequisite. See Open Question 1 in the proofs and the Open Work Items section in the implementation doc.
2. **Implicit transfer $\leftrightarrow$ batch binding.** The chain emits `Transfer` and `Minted` in the same transaction, but no on-chain batch identifier ties the BUCK transfer to the specific `cm` range. Off-chain indexers reconstruct the binding from event correlation. Whether to add an explicit `batchId` storage slot (and a matching public input to the spend circuit) is an open design question; see Open Question 2.
3. **Spend circuit not yet built.** Bob's note is currently not redeemable – there is no `Notes.spend` function. The Phase-7 design will add a Merkle-membership + nullifier + value-conservation SNARK that lets Bob spend his note without revealing which `cm_i` he opened. Until then, the system is "mint-only" and the unlinkability story is an at-rest property of the published commitments rather than an end-to-end chain-observable property of the spend. See Open Question 3 and the Phase 7 and Beyond section.
4. **Phase-6 batch size is pinned at `N=2`.** The walk above used `N=6` for narrative clarity. The shipped `mint.circom` is templated at `N=2` and the `MintVerifierAdapter` hard-codes `MINT_BATCH_SIZE=2`. Splitting 1,200 BUCK into six notes in the Phase-6 deployment requires three sequential `Notes.mint` calls of two notes each (which weakens the unlinkability story

for that particular issuer, since the three calls are linkable as a sequence by `msg.sender`). Variable- N support is deliberately deferred until the spend circuit pins the eventual public-signal arity.

12 Cross-References

- alberta-buck-notes.org – architectural overview of the Notes primitive (flavors, Merkle tree, demurrage interaction).
- alberta-buck-proofs.org – formal correctness arguments (Theorems 7-11; Open Questions in Part IV).
- alberta-buck-ethereum.org – Solidity surface, gas costs, phase-by-phase implementation status, Open Work Items.
- alberta-buck-identity.org – IdentityRegistry, PS credentials, and the bilateral ElGamal handshake that anchors `msg.sender` to a verifiable identity.
- `circuits/mint.circom` – the Phase-6 mint circuit source.
- `src/Notes.sol`, `src/MintGroth16Verifier.sol`, `src/MintVerifierAdapter.sol` – the on-chain code paths walked above.
- `scripts/snark/prove_mint.js` – the reference prover used in tests.
- `test/MintVerifier.t.sol` – the seven Foundry tests (happy path + four tampers + two guards) that pin the soundness story to executable assertions.